

Course code: CSW 306

Course name: Backend Development

Type of Exam: Open book / Practice on Computer

Exam code: 01

Duration: 120'

Maximum mark: 100

Instructions:

- *Non-programming calculator: Yes ☒ | No ☐*
- *This paper consists of 4 questions. This paper carries 100 marks:*
 - Question 1 (30 marks).*
 - Question 2 (30 marks).*
 - Question 3 (30 marks).*
 - Question 4 (10 marks).*

Students must create the project using the naming convention:

Format: `DateTime(DD-MM-YYYY)_BE_YourName`

Example: `18062026_BE_NguyenXuanCuong`

Students zip the project **source code**, **database backup file (*.bak)**, **report docx file** in one compressed file for submission.

Students may refer to the .NET documentation to design and develop ASP.NET Core API web applications using the following:

1. <https://learn.microsoft.com/en-us/aspnet/core/getting-started/?view=aspnetcore-8.0>.
2. **Print or pdf** documents available.

PET STORE MANAGEMENT SOFTWARE

- Company HappyPaws wants to build an internal Web API system to manage pets, pet categories, customers, and pet purchase transactions. You are required to develop a Web API system to fulfill the following requirements.

Question 1: Database, Model and DbContext Design (30 marks)

Task: Design the database schema, entity models, and ApplicationDbContext for the Pet Store Management System.

1. **Classes & Relationships:** The system must include the following entities:

Pet

- A Pet belongs to one PetCategory.
- A PetCategory can have many Pets.

User

- A User has one Role stored as a property.
- Role values: Admin, Customer

PurchaseOrder

- A PurchaseOrder belongs to one User.
- A PurchaseOrder contains many PurchaseItems.

PurchaseItem

- A PurchaseItem belongs to one PurchaseOrder.
- A PurchaseItem belongs to one Pet.

2. Data Annotations/Validation:

Pet

- Name: Required, maximum 100 characters.
- Price: Required, must be greater than 0.

PetCategory

- Name: Required, maximum 50 characters.

User

- FullName: Required, maximum 200 characters.
- Email: Required, valid email format.
- Role: Required.

PurchaseOrder

- OrderDate cannot be a future date.

3. Create ApplicationDbContext: The ApplicationDbContext class must contain the following DbSet:

- DbSet<Pet> Pets
- DbSet<PetCategory> PetCategories
- DbSet<User> Users
- DbSet<PurchaseOrder> PurchaseOrders
- DbSet<PurchaseItem> PurchaseItems

Note: Tables should be created in the database corresponding to the classes.

Table: User

Field	Type	Constraints / Notes
Id	INT	PK, Identity
FullName	NVARCHAR(200)	NOT NULL
Email	NVARCHAR(255)	NOT NULL, UNIQUE
Password	NVARCHAR(MAX)	NOT NULL
Role	NVARCHAR(20)	NOT NULL ('Admin', 'Customer')
CreatedAt	DATETIME	Default GETDATE()

Table: PetCategories

Field	Type	Constraints / Notes
Id	INT	PK, Identity
Name	NVARCHAR(50)	NOT NULL, UNIQUE
Description	NVARCHAR(255)	Optional
CreatedAt	DATETIME	Default GETDATE()

Table: Pets

Field	Type	Constraints / Notes
Id	INT	PK, Identity
Name	NVARCHAR(100)	NOT NULL
Breed	NVARCHAR(100)	NOT NULL
Age	INT	Positive value
Price	DECIMAL(10,2)	NOT NULL, Positive
CategoryId	INT	FK → PetCategories(Id), NOT NULL
Quantity	INT	Default 0
CreatedAt	DATETIME	Default GETDATE()

Table: PurchaseOrders

Field	Type	Constraints / Notes
Id	INT	PK, Identity
UserId	INT	FK → Users(Id), NOT NULL
OrderDate	DATETIME	NOT NULL
TotalAmount	DECIMAL(10,2)	Calculated
Status	NVARCHAR(20)	Pending / Completed / Cancelled
CreatedAt	DATETIME	Default GETDATE()

Table: PurchaseItems

Field	Type	Constraints / Notes
Id	INT	PK, Identity
PurchaseOrderId	INT	FK → PurchaseOrders(Id), NOT NULL
PetId	INT	FK → Pets(Id), NOT NULL
Quantity	INT	Positive value
UnitPrice	DECIMAL(10,2)	Pet price at purchase time
CreatedAt	DATETIME	Default GETDATE()

Question 2: Repository Layer (30 marks)

Implement repository classes that interact with the database using ApplicationDbContext.

Create IPetRepository and PetRepository

Implement the following methods:

- Task<List> GetAll()
- Task<Pet?> GetById(int id)
- Task Add(Pet pet)
- Task Update(Pet pet)
- Task Delete(int id)
- Task<List> GetByCategoryId(int categoryId)

Requirements

- All methods must use ApplicationDbContext.
- Use asynchronous programming (async/await).
- Implement the **User repository layer** as well.
- The remaining classes are not required to implement the repository layer, but they can be implemented similarly if you choose to, following the same structure and pattern.

Question 3: API Controller and Security (30 marks)

Task: Create RESTful APIs for managing Pets and Purchase Orders..

1. PetsController

Method	Route	Description
GET	/api/pets	Get all pets with category information
GET	/api/pets/{id}	Get pet by Id
POST	/api/pets	Create new pet
PUT	/api/pets/{id}	Update pet
DELETE	/api/pets/{id}	Delete pet

Example Response: { "id": 1, "name": "Golden Retriever", "breed": "Retriever", "price": 500, "categoryId": 1, "categoryName": "Dogs" }

2. PurchaseOrdersController

Method	Route	Description
GET	/api/orders	Get all orders with order details
POST	/api/orders	Create a new order
GET	/api/orders/user/{userId}	Get orders by user

Business Rules

When creating an order:

- Extract UserId from JWT token.
- Set OrderDate = current server time.
- Set Status = "Pending".
- Calculate UnitPrice from current Pet.Price.
- Calculate TotalAmount automatically.
- Validate PetId exists.
- Validate Quantity > 0.
- Validate available quantity.

Request Example:

```
{ "items": [{"petId": 1, "quantity": 1}] }
```

3. AuthController

Method	Route	Description
POST	/api/auth/login	Authenticate user and return JWT token

4. PetCategoriesController

Method	Route	Description
GET	/api/categories	Return all pet categories

General Requirements

Validation:

Use: [Required], [Range], [EmailAddress]

Return: 400 Bad Request for invalid data and 404 Not Found for unavailable resources.

Security:

Implement: JWT Authentication, Role-based Authorization

Authorization Rules

Admin can: Create pets, update pets, delete pets, view all orders

Customer can: Create orders, view only their own orders

Guest public can: View pets, view pet categories

Question 4: Project Documentation Report (10 marks)

Prepare a report document (.docx) describing your implementation and testing process.

Section 1: Database Design (2 marks)

Include screenshots of:

- Database tables, Relationships between tables, Sample data.

Provide brief explanations.

Section 2: Project Structure (3 marks)

Include screenshots of:

- Solution Explorer, Models, DbContext, Repositories, Controllers.

Provide brief explanations.

Section 3: API Testing Results (5 marks)

Include screenshots demonstrating:

- Login API, JWT token generation, GET Pets, POST Pet, PUT Pet, DELETE Pet, POST Order.
- Authorization testing (Admin vs Customer).

Each screenshot must clearly display:

- Request URL, HTTP Method, Request Body (if applicable), Response Data.

-----***-----

IMPORTANT NOTES

1. Since there is only one login method in AuthController, you should create the data directly in SQL Server **via** SSMS and use it to log in with the following requirements:

- At least 1 Admin user (Role = "Admin").
- At least 1 Normal user (Role = "Customer").
- Password hashing is optional.

Example users:

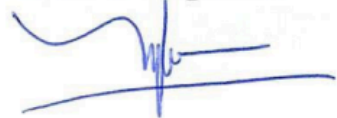
- Email: **admin@test.com**, Password: **123**, Role: **Admin**
- Email: **customer@test.com**, Password: **123**, Role: **Customer**

2. Include these test users in your database backup file submission.

3. Complete the easier tasks first to optimize your time and get maximize score.

-----The End -----

Lecturer's signature

A handwritten signature in blue ink, consisting of a series of loops and a final horizontal stroke.

Cuong Nguyen Xuan

